Lesson 09 Demo 03

Configuring Terraform State Backend Storage

Objective: To configure and manage Terraform state using the AWS S3 backend for ensuring reliable state storage and management

Tools required: Visual Studio Code

Prerequisites: Ensure you have created and implemented the AWS access key and secret key before starting this demo. Refer to Lesson 08 Assisted Practice 02 for detailed steps.

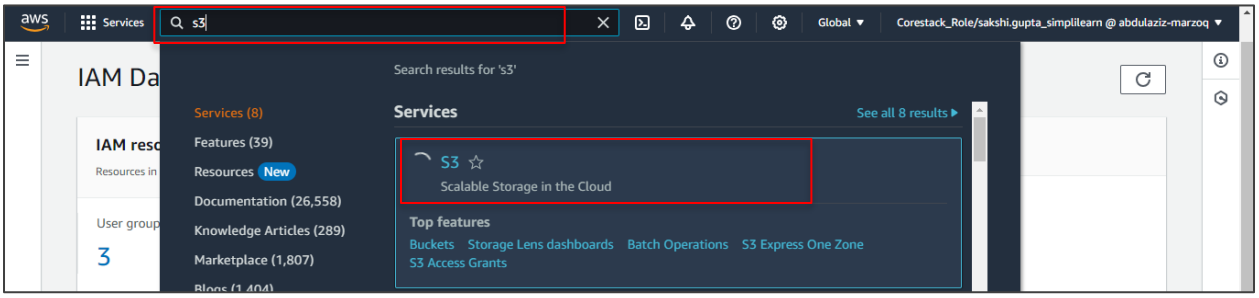
Note: The folder structure created in the previous demos is used here. It is also included in the resources section of LMS. Please refer Lesson_10_demo_01

Steps to be followed:

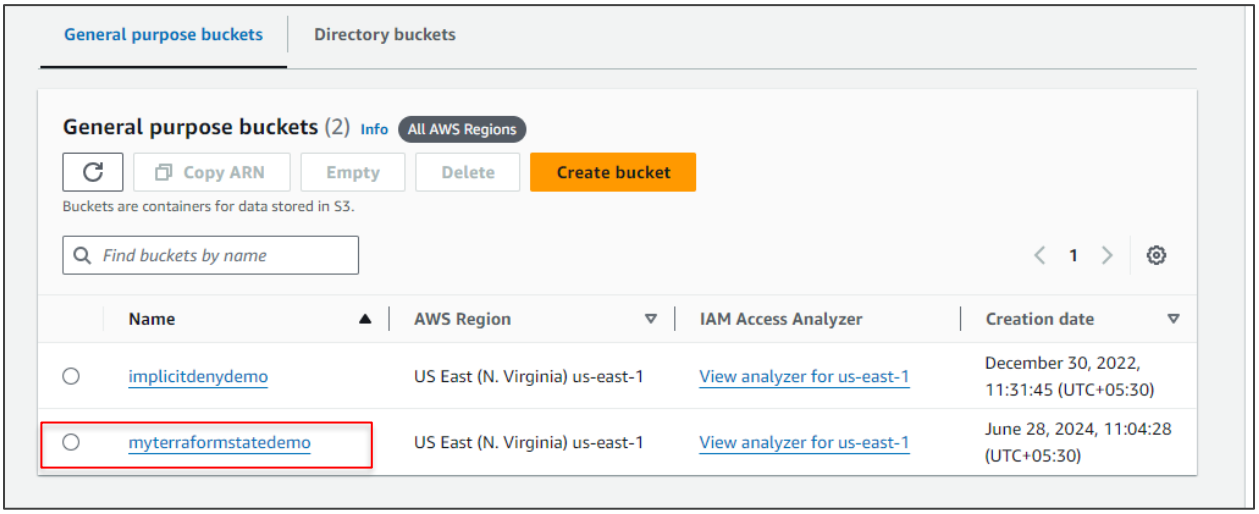
1. Enable versioning on the S3 bucket
2. Enable encryption on the S3 bucket
3. Enable locking for the S3 backend
4. Remove existing resources with the terraform destroy command

Step 1: Enable versioning on the S3 bucket

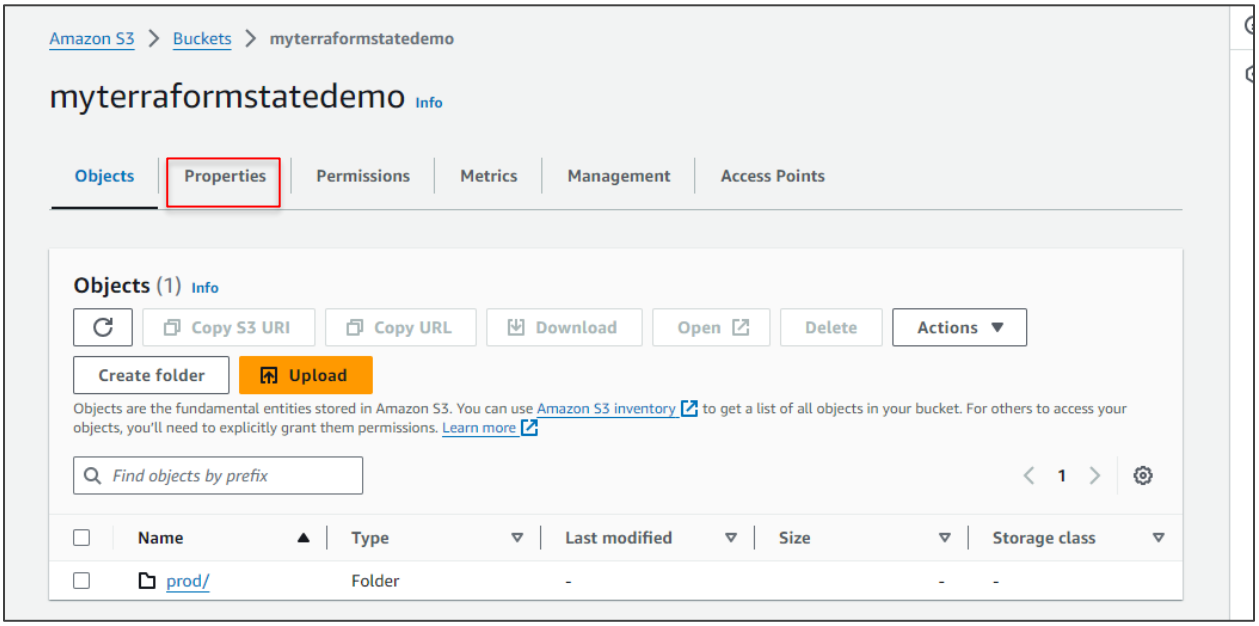
1.1 Log in to the AWS Management Console and navigate to the **S3** service using the search bar

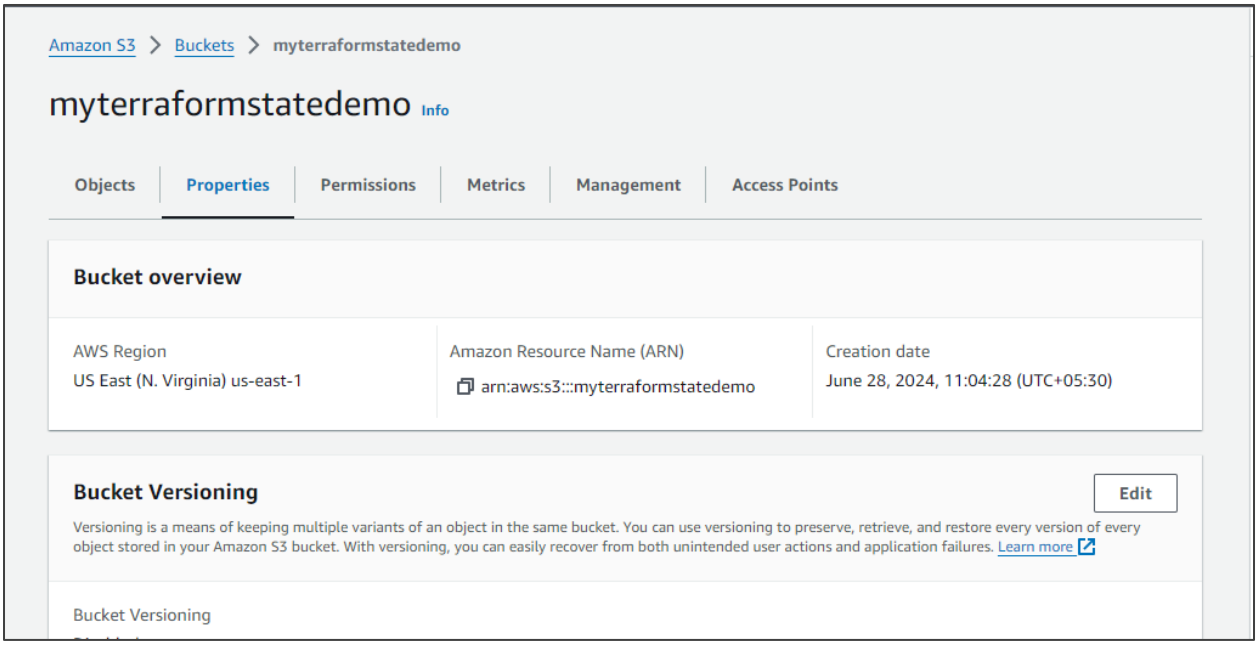


1.2 Click on the previously created bucket named **myterraformstatedemo**

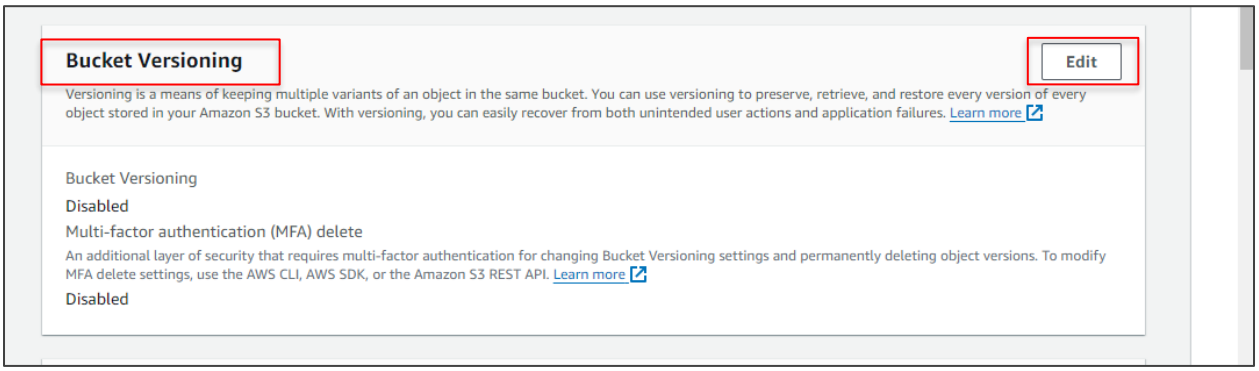


1.3 Click on **Properties**

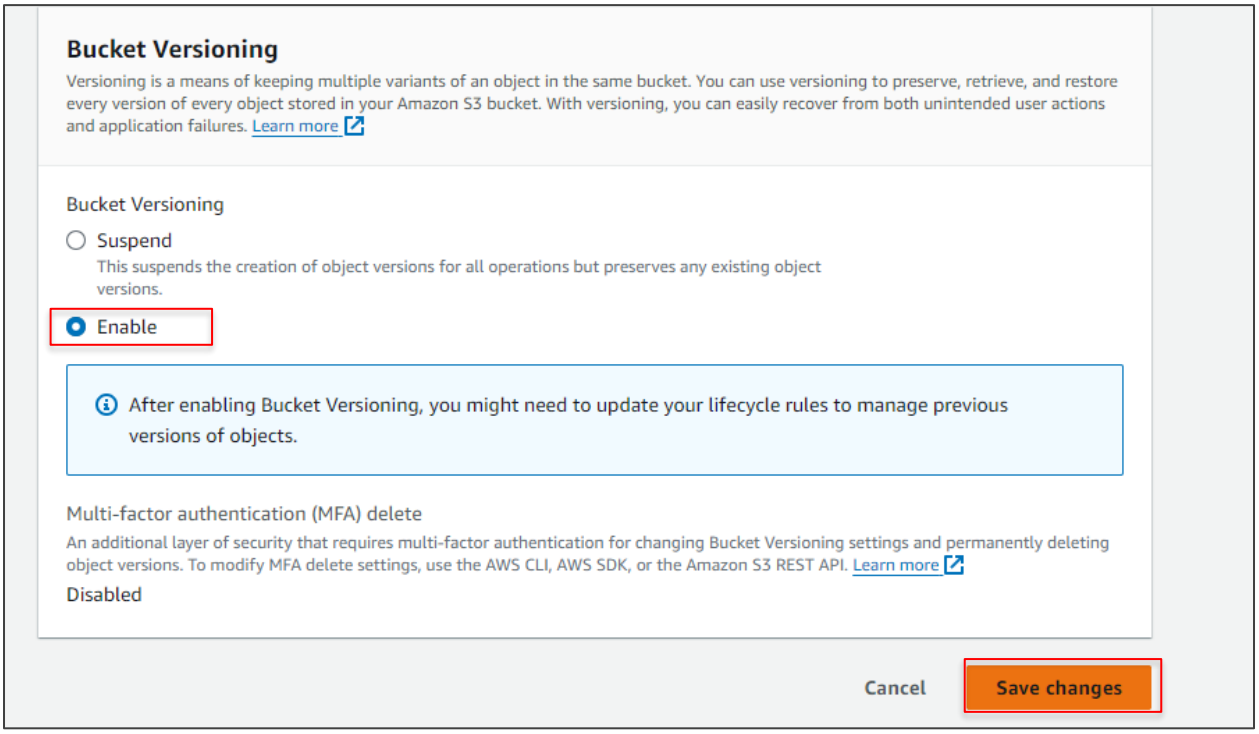




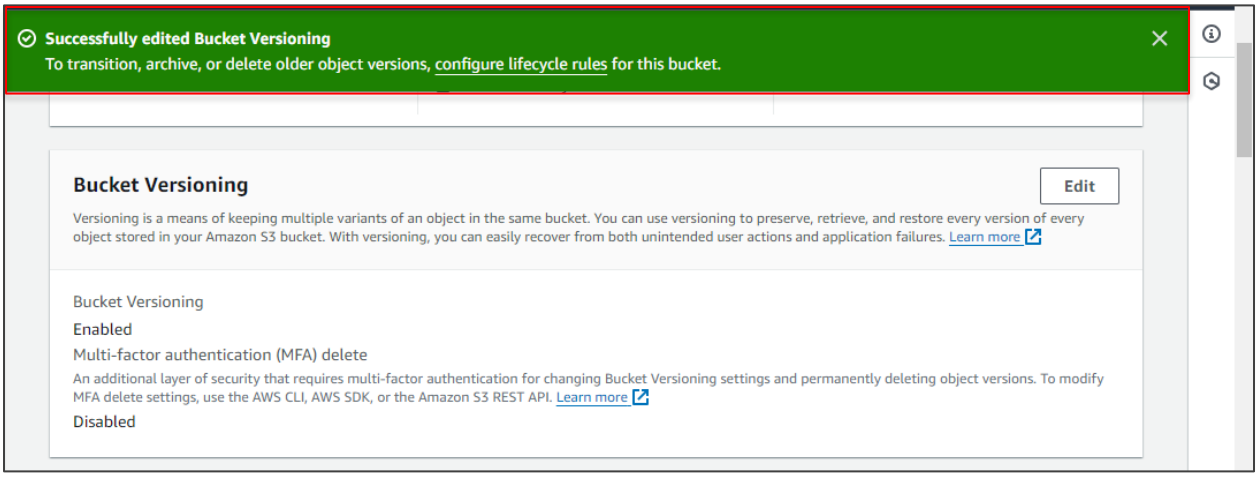
1.4 Scroll down to **Bucket Versioning** and click on **Edit**



1.5 Select **Enable** and click on **Save changes**



Versioning on the S3 bucket is successfully enabled, and the following message is shown:



1.6 Make a configuration change in Terraform **main.tf** file to generate a new state version using the following code:

```
resource "aws_instance" "web_server" {
 ami          = "ami-12345678"
 instance_type = "t2.small"
 subnet_id     = "subnet-12345678"
 tags = {
  Name = "Web EC2 Server 2"
 }
}
```

```
# Terraform Resource Block - To Build EC2 instance in Public Subnet
resource "aws_instance" "web_server_2" {
  ami           = data.aws_ami.ubuntu.id
  instance_type = "t2.small"
  subnet_id     = aws_subnet.public_subnets["public_subnet_2"].id
  tags = {
    Name = "Web EC2 Server 2"
  }
}
```

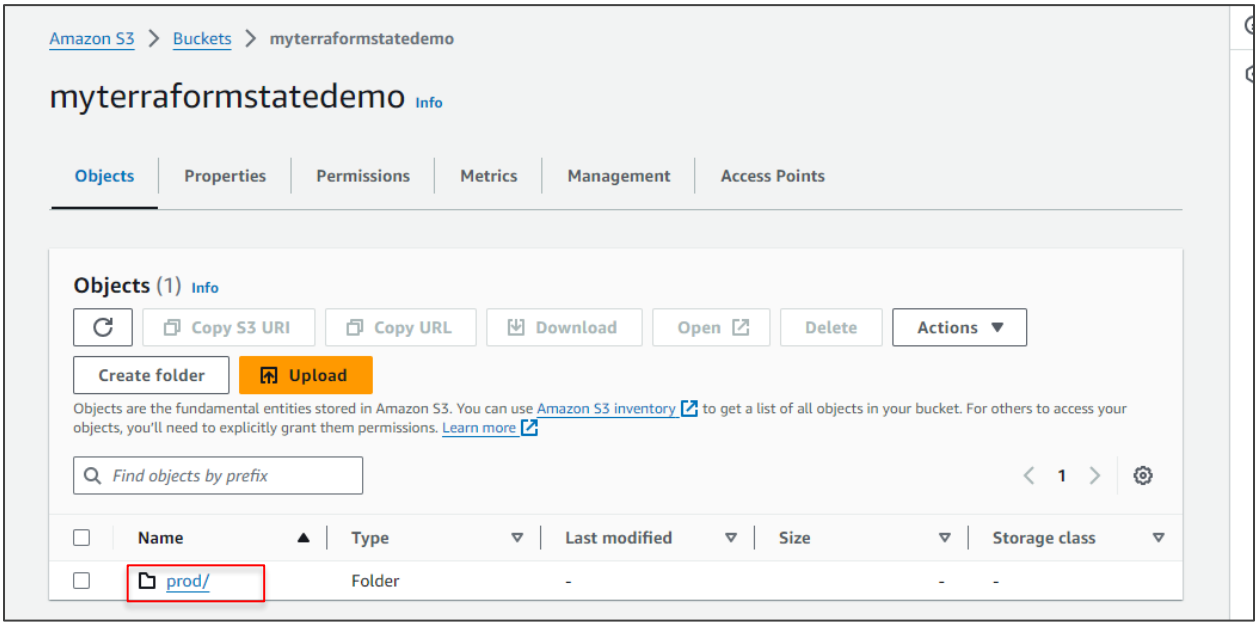
1.7 Execute the following command in the terminal to apply the configuration changes: **terraform apply**

```
sakshiguptasimp@ip-172-31-22-2:~/Desktop/Terraform$ terraform apply
```

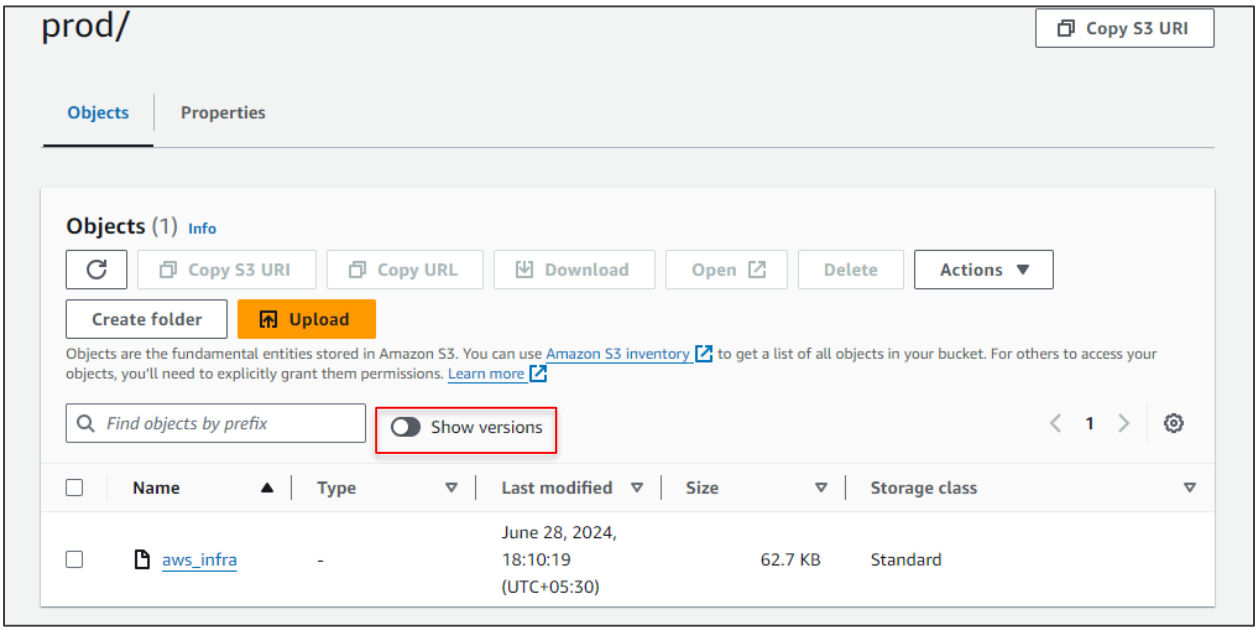
1.8 When prompted, approve the changes by typing **yes**

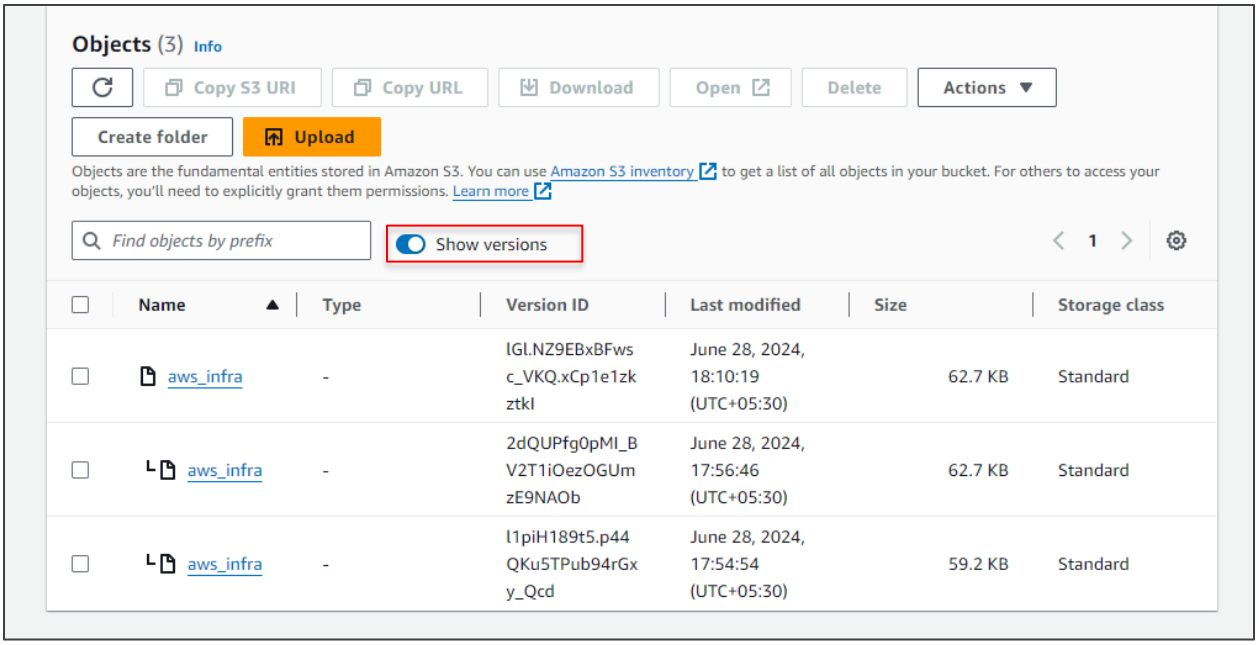
```
PROBLEMS    OUTPUT    TERMINAL    DEBUG CONSOLE    PORTS

    }

Plan: 0 to add, 1 to change, 0 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes
```

1.9 Go to the Amazon S3 management console and click on the folder named **/prod**



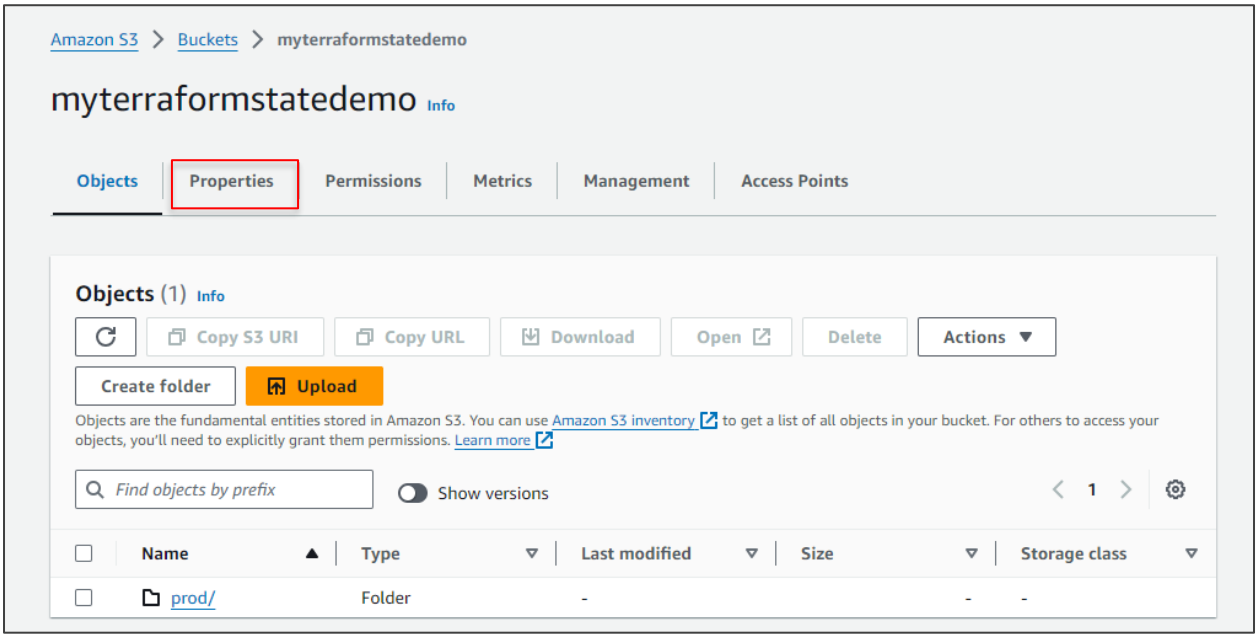
1.10 Enable **Show versions** and the newly created versions of the Terraform configurations will appear



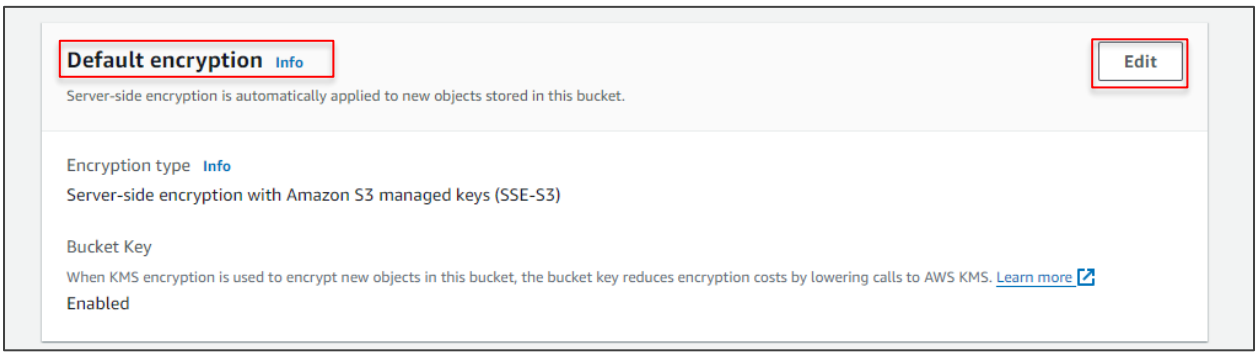


Step 2: Enable encryption on the S3 bucket

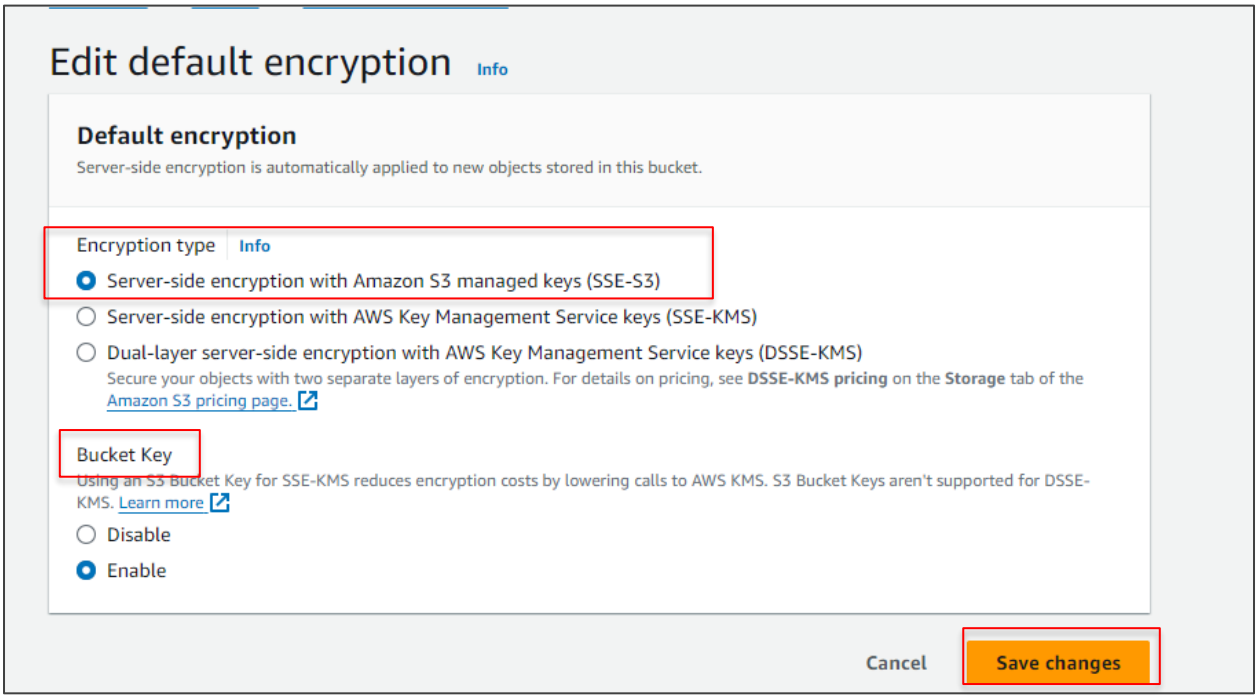
2.1 In the Amazon S3 management console, go into the created bucket named **myterraformstatedemo,** and click on **Properties**



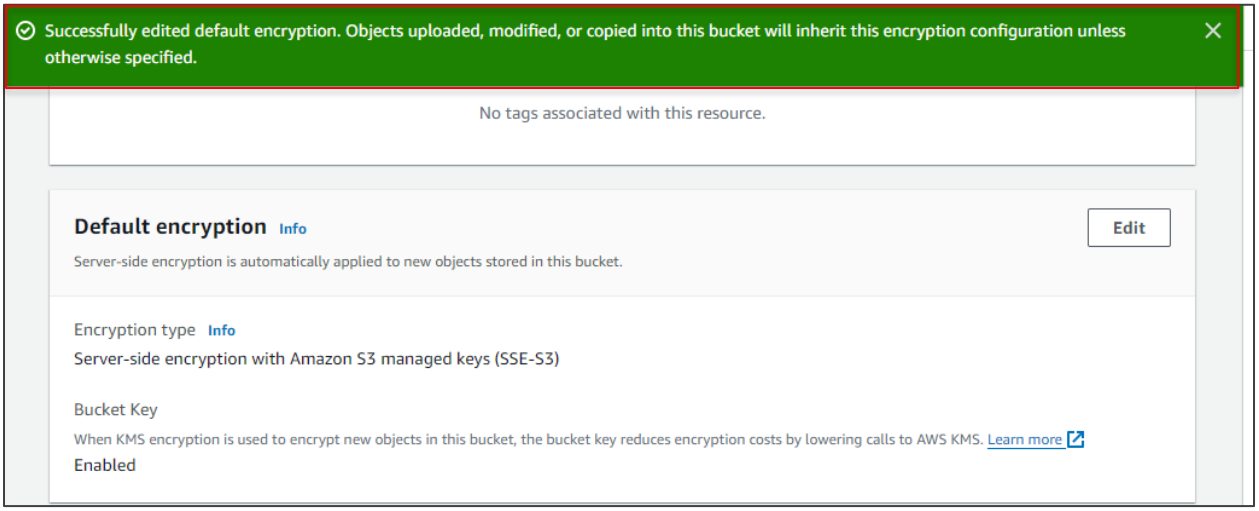
2.2 Scroll down to the **Default encryption** section and click on **Edit**



2.3 Make sure the **Encryption type** is selected and the **Bucket Key** is enabled as shown in the following screenshot. Click on **Save changes**.

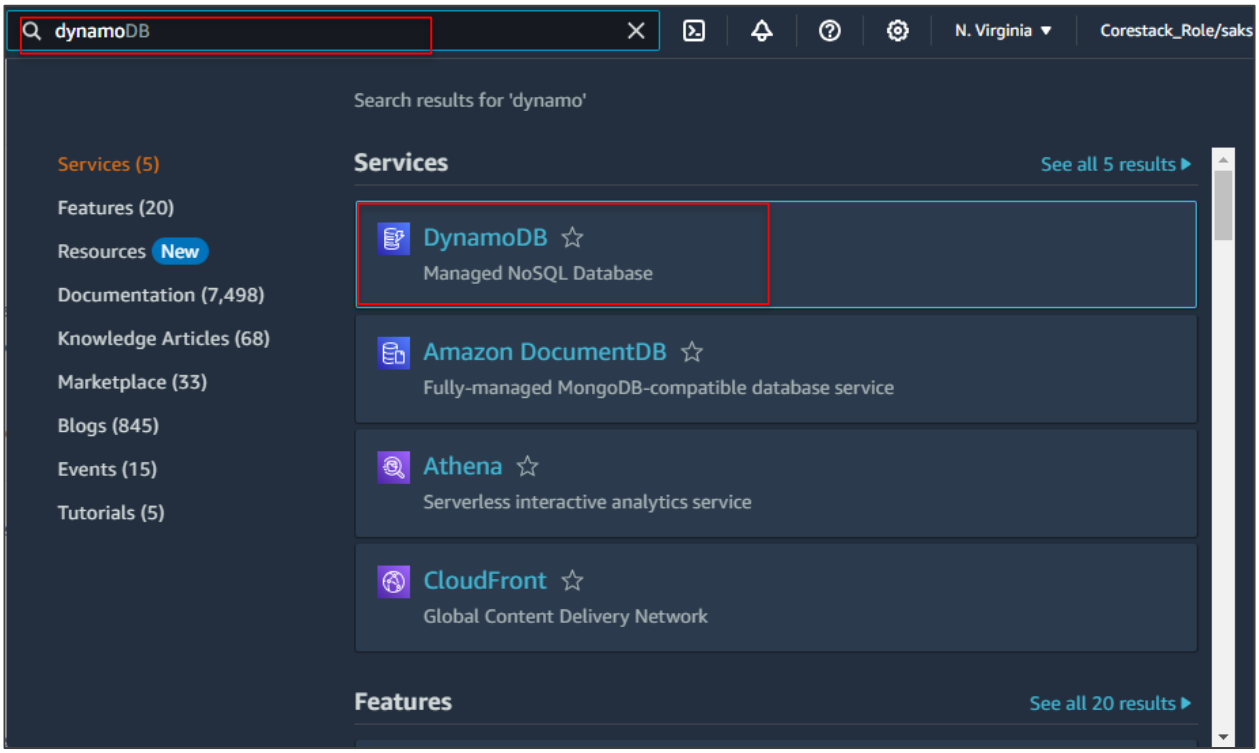


Encryption on the S3 bucket is successfully enabled, and the following message will be displayed:

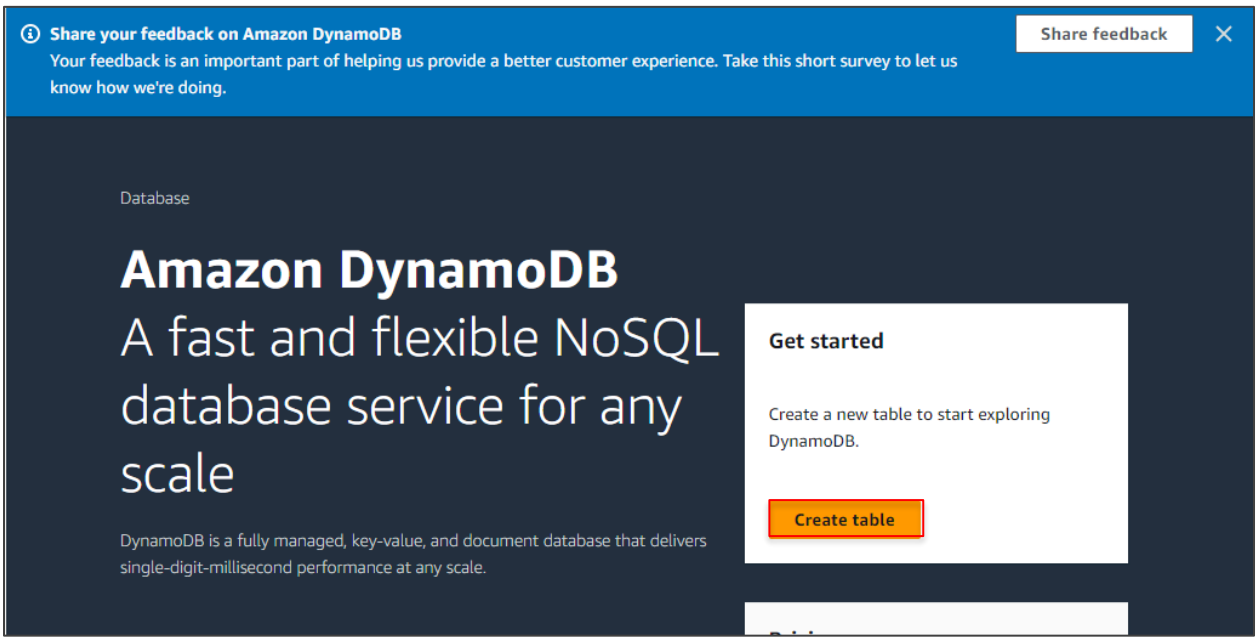


Step 3: Enable locking for the S3 backend

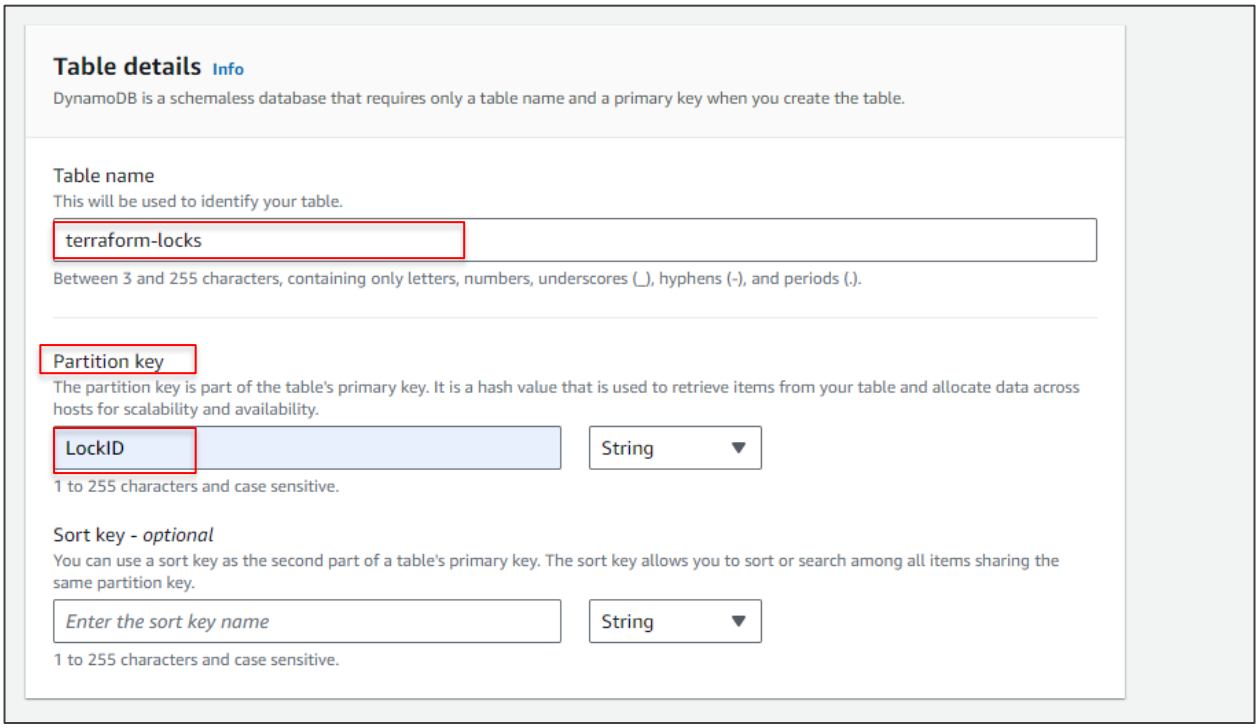
3.1 Go to the AWS Console, search for **dynamoDB** in the search bar, and select the **DynamoDB** service



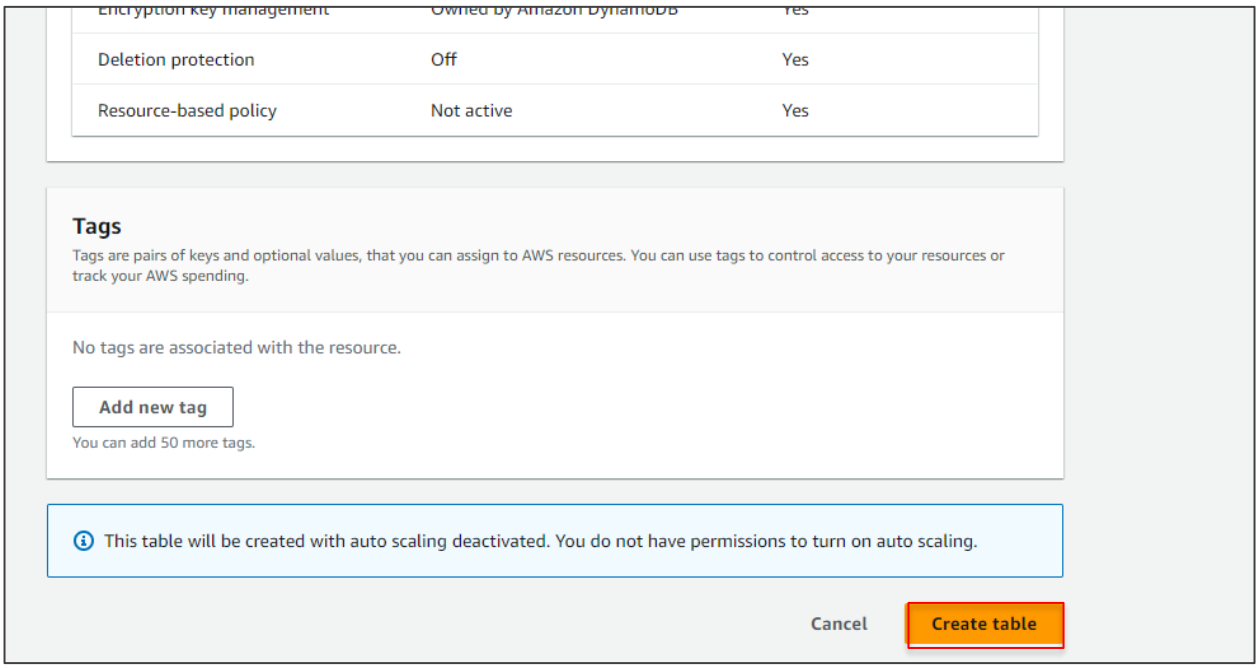
3.2 Click on **Create table**

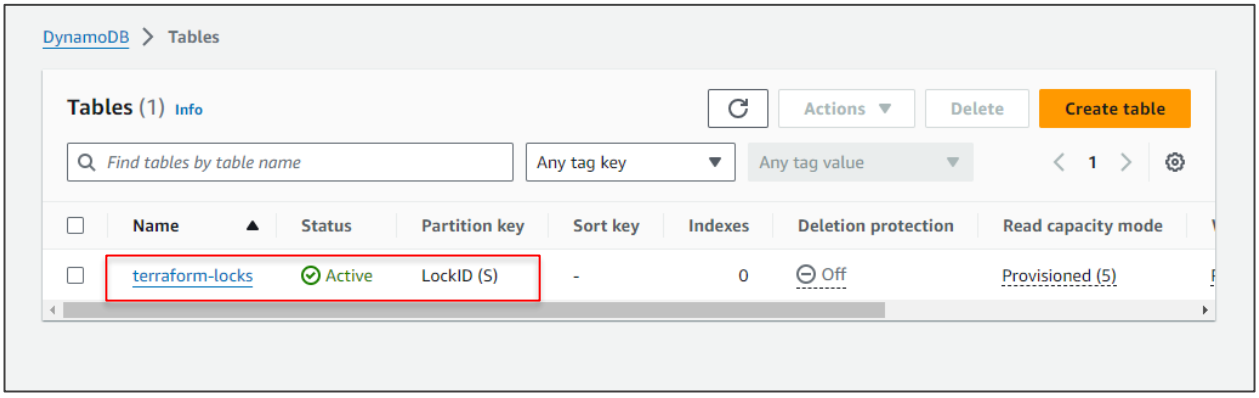


3.3 Name the table as **terraform-locks** and add the **Partition key** as **LockID**



3.4 Retain the other settings as default and click on **Create table**





The table will be created as shown above.

3.5 Update the backend configuration in **terraform.tf** to use the DynamoDB table with the following code:

```
terraform {
  backend "s3" {
    bucket        = "myterraformstatedemo"
    key           = "prod/aws_infra"
    region        = "us-east-1"
    dynamodb_table = "terraform-locks"
    encrypt       = true
  }
}
```

terraform.tf

```
terraform {
  backend "s3" {
    bucket = "myterraformstatedemo"
    key    = "prod/aws_infra"
    region = "us-east-1"

    # Replace this with your DynamoDB table name!
    dynamodb_table = "terraform-locks"
    encrypt = true
  }

```

3.6 Reinitialize the backend using the following command:

terraform init -reconfigure

```
sakshiguptasimp@ip-172-31-22-2:~/Desktop/Terraform$ terraform init -reconfigure
Initializing modules...

Initializing the backend...

Successfully configured the backend "s3"! Terraform will automatically
use this backend unless the backend configuration changes.

Initializing provider plugins...
```

```
You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
sakshiguptasimp@ip-172-31-22-2:~/Desktop/Terraform$
```

3.7 Use the following command to apply the configuration changes:

terraform apply

```
sakshiguptasimp@ip-172-31-22-2:~/Desktop/Terraform$ terraform apply
```

3.8 When prompted, approve the changes by typing **yes**

```
PROBLEMS    OUTPUT    TERMINAL    DEBUG CONSOLE    PORTS

    }

Plan: 0 to add, 1 to change, 0 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes
```

```
PROBLEMS    OUTPUT    TERMINAL    DEBUG CONSOLE    PORTS

Outputs:

public_dns = "ec2-18-207-104-133.compute-1.amazonaws.com"
public_dns_server_subnet_1 = "ec2-54-163-63-185.compute-1.amazonaws.com"
public_ip = "18.207.104.133"
public_ip_server_subnet_1 = "54.163.63.185"
size = "t2.micro"
sakshiguptasimp@ip-172-31-22-2:~/Desktop/Terraform$
```

Step 4: Remove existing resources with the terraform destroy command

4.1 Clean up and destroy all managed infrastructure to prepare for the next demo using the following command:
terraform destroy

```
PROBLEMS    OUTPUT    TERMINAL    DEBUG CONSOLE    PORTS

public_ip_server_subnet_1 = "54.163.63.185"
size = "t2.micro"
sakshiguptasimp@ip-172-31-22-2:~/Desktop/Terraform$ terraform destroy
random_string.random: Refreshing state... [id=aFH6wa#z4]]
tls_private_key.generated: Refreshing state... [id=97e6e3926721ef3e34a143c66d76d2db03b1caa9]
local_file.private_key_pem: Refreshing state... [id=a373ae1b341cce8b6650e7be99b70d875bdf9f74]
aws_key_pair.generated: Refreshing state... [id=MyAWSKey]
aws_vpc.vpc: Refreshing state... [id=vpc-0a22392922397513a]
```

4.2 When prompted, approve the destroy by typing **yes**

```
PROBLEMS    OUTPUT    TERMINAL    DEBUG CONSOLE    PORTS

  - public_ip                  = "18.207.104.133" -> null
  - public_ip_server_subnet_1  = "54.163.63.185" -> null
  - size                       = "t2.micro" -> null

Do you really want to destroy all resources?
  Terraform will destroy all your managed infrastructure, as shown above.
  There is no undo. Only 'yes' will be accepted to confirm.

  Enter a value:
```

```
PROBLEMS    OUTPUT    TERMINAL    DEBUG CONSOLE    PORTS

  - public_ip                  = "18.207.104.133" -> null
  - public_ip_server_subnet_1  = "54.163.63.185" -> null
  - size                       = "t2.micro" -> null

Do you really want to destroy all resources?
  Terraform will destroy all your managed infrastructure, as shown above.
  There is no undo. Only 'yes' will be accepted to confirm.

  Enter a value: yes
```

```
aws_eip.nat_gateway_eip: Destruction complete after 1s
aws_internet_gateway.internet_gateway: Destroying... [id=igw-05730369c684d5b08]
aws_internet_gateway.internet_gateway: Destruction complete after 0s
aws_vpc.vpc: Destroying... [id=vpc-0a22392922397513a]
aws_vpc.vpc: Destruction complete after 0s

Destroy complete! Resources: 30 destroyed.
```

4.3 Validate the destruction using the following command:
terraform state list

```
sakshiguptasimp@ip-172-31-22-2:~/Desktop/Terraform$ terraform state list
sakshiguptasimp@ip-172-31-22-2:~/Desktop/Terraform$
```

Since there are no files present, this indicates that the infrastructure has been successfully cleaned and destroyed.

By following these steps, you have successfully configured and managed Terraform state using the AWS S3 backend for ensuring reliable state storage and management.